

Java2 Day05 핵심 정리

애너테이션, enum, 재귀호출, DI를 처음 배우는 사람도 따라갈 수 있게 정리한 자료

주제: 애너테이션, enum, 재귀호출, DI

대상 독자: Java2 Day05 내용을 처음 복습하는 초보자

작성 기준 자료: Day5 TXT, Day5 원본 PDF, java2prjDay5 day5 소스 코드

생성일: 2026-05-21

애너테이션

코드에 설명표를 붙이고, 실행 중에 그 정보를 읽어 검증한다.

enum

정해진 값만 사용하게 해서 오타와 잘못된 값을 막는다.

재귀호출

메서드가 자기 자신을 다시 호출해서 문제를 작게 나누어 해결한다.

DI

필요한 객체를 직접 만들지 않고 외부에서 넣어 받아 변경에 강한 구조를 만든다.

0. 목차와 학습 로드맵

단원	학습 내용
1	Java 언어 기본 용어
2	애너테이션과 리플렉션 검증
3	enum 열거형
4	재귀호출
5	DI, 의존성 주입
6	오후 개인 실습 풀이 방향
7~9	연습 문제, 확인 불가 항목, 최종 요약

1. 먼저 클래스, 객체, 필드, 메서드 같은 기본 용어를 확인합니다.
2. 애너테이션은 "코드에 붙이는 정보"로 이해합니다.
3. enum은 "정해진 값만 허용하는 타입"으로 이해합니다.
4. 재귀호출은 종료 조건을 먼저 찾는 습관을 들입니다.
5. DI는 직접 생성하지 않고 외부에서 넣어 받는 구조로 정리합니다.

원본 자료	확인 내용	반영 방식
5차시.txt	전체 내용 확인	학습 목표와 오후 개인 실습 반영
Day5 원본 PDF 5개	파일 존재, PDF 헤더, 페이지 구조 확인	애너테이션, 재귀호출 주제 반영
workspace/java2prjDay5/src/day5	79개 파일 구조와 대표 예제 확인	annotation, enumEx, 재귀호출, di_injection 코드 예제로 반영

확인 제한: PDF 본문 세부 텍스트는 완전 추출하지 못했습니다. TXT 전체 내용과 실제 소스 코드 확인 내용을 중심으로 재구성했습니다.

1. Java 언어 기본부터 보기

Java는 프로그램을 **클래스(class)** 단위로 작성하는 객체지향 언어입니다. 클래스 안에는 값이나 상태를 저장하는 **필드(field)**, 일을 처리하는 **메서드(method)**, 객체를 만들 때 실행되는 **생성자(constructor)**가 들어갑니다.

용어	뜻	간단한 예
클래스	객체를 만들기 위한 설계도	<code>class Phone { ... }</code>
객체	클래스로 실제 생성한 것	<code>new Phone()</code>
필드	객체가 가지고 있는 데이터	<code>String name;</code>
메서드	객체가 할 수 있는 동작	<code>void powerOn()</code>
인터페이스	반드시 구현해야 할 기능의 약속	<code>interface Battery</code>

Day05에서 중요한 관점은 “코드를 그냥 실행하는 것”에서 한 단계 더 나아가, 코드에 정보를 붙이고, 타입을 제한하고, 반복 구조를 다르게 생각하고, 객체 사이의 관계를 느슨하게 만드는 것입니다.

2. 애너테이션 Annotation

애너테이션은 코드에 붙이는 **표식**입니다. Java에서는 `@Override`, `@Deprecated`, `@SuppressWarnings` 같은 애너테이션을 이미 제공합니다. Day05에서는 직접 애너테이션을 만들고, 리플렉션으로 읽어서 검증하는 방법을 배웁니다.

2-1. 애너테이션 유지 범위

유지 범위는 애너테이션 정보가 어디까지 남아 있는지를 정합니다. **리플렉션으로 실행 중에 읽고 싶다면 반드시 RUNTIME**을 사용해야 합니다.

RetentionPolicy	뜻	언제 쓰나
SOURCE	소스 코드에만 존재하고 컴파일 후 사라짐	컴파일러 확인용
CLASS	class 파일까지 남지만 실행 중 리플렉션으로는 보통 읽지 않음	바이트코드 도구용
RUNTIME	실행 중에도 남아 리플렉션으로 읽을 수 있음	검증, 설정, 프레임워크 처리

```
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;

@Retention(RetentionPolicy.RUNTIME)
public @interface Count {
    int value() default 1;
}
```

위 코드는 @Count라는 애너테이션을 직접 만든 예입니다. value()는 애너테이션에 넣을 값을 의미합니다. 이름이 value일 때는 @Count(value = 3) 대신 @Count(3)처럼 줄여 쓸 수 있습니다.

2-2. 애너테이션을 필드에 붙이기

```
public class FruitStore {
    @Count(3)
    private int apples;

    @Count(5)
    private int bananas;

    @Count(10)
    private int tomato;

    public FruitStore(int apples, int bananas, int tomato) {
        this.apples = apples;
        this.bananas = bananas;
        this.tomato = tomato;
    }
}
```

여기서 @Count(3)은 “apples는 3개여야 한다”는 의미로 사용됩니다. 애너테이션 자체는 값을 검사하지 않습니다. **검사하는 코드는 따로 작성해야 합니다.**

2-3. 리플렉션으로 애너테이션 읽기

```
import java.lang.reflect.Field;

public class ValidationUtils {
    public static void validate(Object obj) throws IllegalAccessException {
        Class<?> clazz = obj.getClass();
        Field[] fields = clazz.getDeclaredFields();

        for (Field field : fields) {
            Count count = field.getAnnotation(Count.class);

            if (count != null) {
                field.setAccessible(true);
                int realValue = (int) field.get(obj);
                int expectedValue = count.value();

                if (realValue != expectedValue) {
                    throw new IllegalArgumentException(
                        field.getName() + " 항목은 " + expectedValue + " 값이어야 합니다."
                    );
                }
            }
        }
    }
}
```

1. obj.getClass()로 객체의 클래스 정보를 가져옵니다.
2. getDeclaredFields()로 클래스 안의 필드 목록을 가져옵니다.
3. field.getAnnotation(Count.class)로 해당 필드에 붙은 @Count를 찾습니다.
4. field.setAccessible(true)로 private 필드도 읽을 수 있게 합니다.
5. 실제 값과 애너테이션에 적힌 기대 값을 비교합니다.

핵심: 애너테이션은 "규칙을 적는 곳"이고, 리플렉션 검증 코드는 "규칙을 실제로 확인하는 곳"입니다.

2-4. @Target으로 붙일 위치 제한하기

```
import java.lang.annotation.ElementType;
import java.lang.annotation.Retention;
import java.lang.annotation.RetentionPolicy;
import java.lang.annotation.Target;

@Target(ElementType.FIELD)
@Retention(RetentionPolicy.RUNTIME)
public @interface MaxLength {
    int value() default 10;
}
```

@Target(ElementType.FIELD)는 이 애너테이션을 필드에만 붙일 수 있다는 뜻입니다. 메서드에 붙이고 싶으면 ElementType.METHOD, 클래스에 붙이고 싶으면 ElementType.TYPE를 사용합니다.

3. enum 열거형

enum은 사용할 수 있는 값을 **정해진 선택지로 제한**하는 문법입니다. 문자열을 직접 쓰면 "LIGHT"를 "LIGTH"처럼 잘못 써도 컴파일 단계에서 잡기 어렵습니다. enum을 쓰면 정해진 값 외에는 넣을 수 없어서 안전합니다.

```
public enum Fruit {
    APPLE, BANANA
}
```

```
public class Program {
    private Fruit fruit;

    public void setFruit(Fruit fruit) {
        this.fruit = fruit;
    }

    public void printFruit() {
        System.out.println(fruit);
    }

    public static void main(String[] args) {
        Program p = new Program();
        p.setFruit(Fruit.BANANA);
        p.printFruit();
    }
}
```

enum 값은 관례적으로 **대문자**로 작성합니다. 예: APPLE, BANANA, MONDAY, DARK

3-1. 문자열과 enum 비교

```
// 문자열 방식: 오타가 들어갈 수 있음
String mode = "LIGHT";
mode = "lite"; // 컴파일 에러가 아님
```

```
// enum 방식: 정해진 값만 가능
Mode mode2 = Mode.LIGHT;
// mode2 = "lite"; // 컴파일 에러
```

```
public enum Mode {
    DARK,
    LIGHT
}
```

enum을 사용하면 잘못된 값이 들어가는 일을 컴파일 단계에서 막을 수 있습니다. 이것이 enum의 가장 큰 장점입니다.

3-2. enum에 한글 이름 붙이기

```
public enum WeekDay {
    MON("월"), TUE("화"), WED("수"), THU("목"), FRI("금");

    private String name;

    WeekDay(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

enum도 클래스처럼 필드와 생성자, 메서드를 가질 수 있습니다. 위 코드는 WeekDay.MON이라는 enum 값에 화면 출력용 이름 "월"을 연결한 예입니다.

3-3. enum을 switch에서 사용하기

```
enum DiceFace {
    ONE, TWO, THREE, FOUR, FIVE, SIX
}

DiceFace diceFace = DiceFace.THREE;

switch (diceFace) {
    case ONE:
        System.out.println("한 개");
        break;
    case TWO:
        System.out.println("두 개");
        break;
    case THREE:
        System.out.println("세 개");
        break;
    default:
        System.out.println("그 외");
}
```

enum은 switch와 잘 어울립니다. 선택지가 정해져 있으므로 분기 처리가 명확해집니다.

4. 재귀호출 Recursive Call

재귀호출은 메서드가 **자기 자신을 다시 호출**하는 방식입니다. 반복문으로 풀 수 있는 문제 중 일부는 재귀로도 풀 수 있습니다.

재귀의 필수 조건: 반드시 종료 조건이 있어야 합니다. 종료 조건이 없으면 메서드가 끝없이 자기 자신을 호출하다가 오류가 발생합니다.

4-1. 1부터 n까지 합 구하기

```
public static int getTotalR(int n) {
    if (n == 1) {
        return 1;
    }

    return n + getTotalR(n - 1);
}
```

getTotalR(5)는 다음처럼 계산됩니다.

```
getTotalR(5)
= 5 + getTotalR(4)
= 5 + 4 + getTotalR(3)
= 5 + 4 + 3 + getTotalR(2)
= 5 + 4 + 3 + 2 + getTotalR(1)
= 5 + 4 + 3 + 2 + 1
= 15
```

여기서 if (n == 1)이 종료 조건입니다. 이 조건이 없으면 n이 계속 작아지면서 끝나지 않습니다.

4-2. 별 출력 재귀 예제

```
public static void printStars(int n) {
    if (n == 0) {
        return;
    }

    printStars(n - 1);
    System.out.println("*");
}
```

printStars(5)를 호출하면 별이 5번 출력됩니다. 먼저 printStars(4), printStars(3)처럼 계속 내려갔다가, n == 0에서 멈춘 뒤 다시 돌아오면서 별을 출력합니다.

4-3. 배열 최대값 찾기

```
private static int findMax(int[] arr, int start, int end) {
    if (start == end) {
        return arr[start];
    }

    int mid = (start + end) / 2;
    int max1 = findMax(arr, start, mid);
    int max2 = findMax(arr, mid + 1, end);

    return max1 > max2 ? max1 : max2;
}
```

이 코드는 배열을 반으로 나누고, 왼쪽 최대값과 오른쪽 최대값을 각각 재귀로 찾은 다음 둘 중 큰 값을 반환합니다. 문제를 작게 나누어 해결하는 전형적인 재귀 방식입니다.

5. DI Dependency Injection

DI는 **의존성 주입**이라는 뜻입니다. 어떤 클래스가 필요한 객체를 직접 만들지 않고, 외부에서 넣어 받는 방식입니다.

5-1. 의존성이란?

Phone이 전원을 켜려면 배터리가 필요합니다. 즉, Phone은 Battery에 의존합니다.

```
public interface Battery {  
    int getEnergy();  
}
```

```
public class LGBattery implements Battery {  
    @Override  
    public int getEnergy() {  
        System.out.println("LG 배터리 에너지 가져오기");  
        return 1000;  
    }  
}
```

```
public class SMBattery implements Battery {  
    @Override  
    public int getEnergy() {  
        System.out.println("SM 배터리 에너지 가져오기");  
        return 2000;  
    }  
}
```

LGBattery와 SMBattery는 모두 Battery 인터페이스를 구현합니다. 그래서 Phone은 구체적인 배터리 이름을 몰라도 Battery 타입으로 사용할 수 있습니다.

5-2. setter 주입과 생성자 주입

```
public class Phone {
    private Bettery bettery;

    // setter 주입
    public void setBettery(Bettery bettery) {
        this.bettery = bettery;
    }

    // 생성자 주입
    public Phone(Bettery bettery) {
        this.bettery = bettery;
    }

    public Phone() {
    }

    public void powerOn() {
        bettery.getEnergy();
        System.out.println("파워가 켜집니다");
    }
}
```

```
// setter 주입
Phone phone1 = new Phone();
phone1.setBettery(new LGBettery());
phone1.powerOn();

// 생성자 주입
Phone phone2 = new Phone(new SMBettery());
phone2.powerOn();
```

방식	뜻	특징
setter 주입	객체를 만든 뒤 setter로 필요한 객체를 넣음	중간에 바꿀 수 있음
생성자 주입	객체를 만들 때 생성자로 필요한 객체를 넣음	처음부터 필요한 값이 반드시 들어감

DI의 핵심: Phone이 new LGBettery()를 직접 하지 않게 만드는 것입니다. 그러면 배터리 구현체가 바뀌어도 Phone 코드를 덜 수정하게 됩니다.

5-3. 설정 파일로 구현체 바꾸기

```
// config.txt
Bettery=day5.di_injection.LGBettery
```

```

Properties p = new Properties();
p.load(new FileReader("src/day5/di_injection/config.txt"));

String className = p.getProperty("Bettery");
Class clazz = Class.forName(className);
Bettery bettery = (Bettery) clazz.newInstance();

Phone phone = new Phone();
phone.setBettery(bettery);
phone.powerOn();

```

설정 파일의 클래스명만 바꾸면 Java 코드를 직접 수정하지 않고도 다른 배터리 구현체를 사용할 수 있습니다. 이것이 변경에 유리한 구조입니다.

6. 오후 개인 실습 풀이 방향

6-1. 별 5개를 재귀 메서드로 출력

```

public class StarPractice {
    public static void printStar(int n) {
        if (n == 0) {
            return;
        }

        System.out.println("*");
        printStar(n - 1);
    }

    public static void main(String[] args) {
        printStar(5);
    }
}

```

n은 남은 출력 횟수입니다. 별을 하나 출력하고 n - 1을 넘기면, 결국 n == 0이 되어 종료됩니다.

6-2. BookGenre enum과 Book 클래스

```
public enum BookGenre {
    NOVEL("소설"),
    HISTORY("역사"),
    SCIENCE("과학"),
    SELF_DEVELOPMENT("자기계발");

    private final String name;

    BookGenre(String name) {
        this.name = name;
    }

    public String getName() {
        return name;
    }
}
```

```
public class Book {
    private String title;
    private String author;
    private BookGenre genre;

    public Book(String title, String author, BookGenre genre) {
        this.title = title;
        this.author = author;
        this.genre = genre;
    }

    public void printInfo() {
        System.out.println("제목: " + title);
        System.out.println("저자: " + author);
        System.out.println("장르: " + genre.getName());
    }

    public static void main(String[] args) {
        Book book = new Book("자바의 정석", "남궁성", BookGenre.SCIENCE);
        book.printInfo();
    }
}
```

장르를 문자열로 쓰지 않고 BookGenre enum으로 받으면, 장르에 정해진 값만 넣을 수 있습니다.

6-3. 사용자 애너테이션 2개 만들기

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface Required {
}
```

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface MinLength {
    int value();
}
```

@Required는 값이 반드시 있어야 한다는 표시이고, @MinLength는 문자열의 최소 길이를 표시하는 애너테이션입니다. 둘 다 실행 중에 읽어야 하므로 **RetentionPolicy.RUNTIME**을 사용합니다.

6-4. Drink와 @MaxSugar 검증

```
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
public @interface MaxSugar {
    int value();
}
```

```
public class Drink {
    private String name;

    @MaxSugar(30)
    private int sugar;

    public Drink(String name, int sugar) {
        this.name = name;
        this.sugar = sugar;
    }
}
```

```

import java.lang.reflect.Field;

public class DrinkValidator {
    public static void validate(Object obj) throws IllegalAccessException {
        Field[] fields = obj.getClass().getDeclaredFields();

        for (Field field : fields) {
            MaxSugar maxSugar = field.getAnnotation(MaxSugar.class);

            if (maxSugar != null) {
                field.setAccessible(true);
                int realSugar = (int) field.get(obj);
                int limit = maxSugar.value();

                if (realSugar > limit) {
                    throw new IllegalArgumentException("당분 과다");
                }
            }
        }
    }

    public static void main(String[] args) throws IllegalAccessException {
        Drink drink = new Drink("초코라떼", 45);
        validate(drink);
    }
}

```

@MaxSugar(30)은 sugar 필드의 최대 허용값이 30이라는 뜻입니다. 검증 코드에서 실제 sugar 값이 30보다 크면 **당분 과다**로 실패시킵니다.

7. 연습 문제와 정답 해설

문제

1. 실행 중 리플렉션으로 읽을 수 있는 애너테이션 유지 범위는 무엇인가?
2. enum을 사용하는 가장 큰 이유는 무엇인가?
3. 재귀호출에서 반드시 필요한 조건은 무엇인가?
4. DI에서 객체가 필요한 의존 객체를 직접 만들지 않는 이유는 무엇인가?

정답 및 해설

번호	정답 해설
1	RetentionPolicy.RUNTIME입니다. 실행 중에도 애너테이션 정보가 남아 있어야 리플렉션으로 읽을 수 있습니다.
2	정해진 값만 사용하게 해서 오타와 잘못된 값 입력을 막기 위해서입니다.
3	종료 조건입니다. 종료 조건이 없으면 자기 자신을 끝없이 호출합니다.
4	구현체가 바뀌어도 기존 코드 수정이 적어지고 변경에 유리해지기 때문입니다.

8. 원본 출처와 확인 불가 항목

항목	이유	필요한 추가 자료
Day5 원본 PDF 세부 본문 전체 추출	PDF 텍스트 추출 도구 없음	텍스트 추출본 또는 OCR 결과
코드 실행 결과	이번 자료 제작에서는 실행 검증하지 않음	Eclipse 또는 javac 실행 환경 확인

9. 마지막 핵심 정리

주제	반드시 기억할 말
애너테이션	코드에 붙이는 정보 이며, 실행 중 읽으려면 RUNTIME이 필요하다.
리플렉션	실행 중 클래스, 필드, 메서드 정보를 읽고 private 값도 접근할 수 있다.
enum	정해진 값만 쓰게 하는 타입 이다. 오타와 잘못된 값을 막는다.
재귀호출	자기 자신을 호출한다. 종료 조건이 없으면 안 된다.
DI	필요한 객체를 외부에서 넣어 받는다. 구현체 변경에 유리하다.